Question

Write a Java program to demonstrate the concepts of Object-Oriented Programming (Encapsulation, Inheritance, Polymorphism, Abstraction, and Interfaces) using a Library Management System example

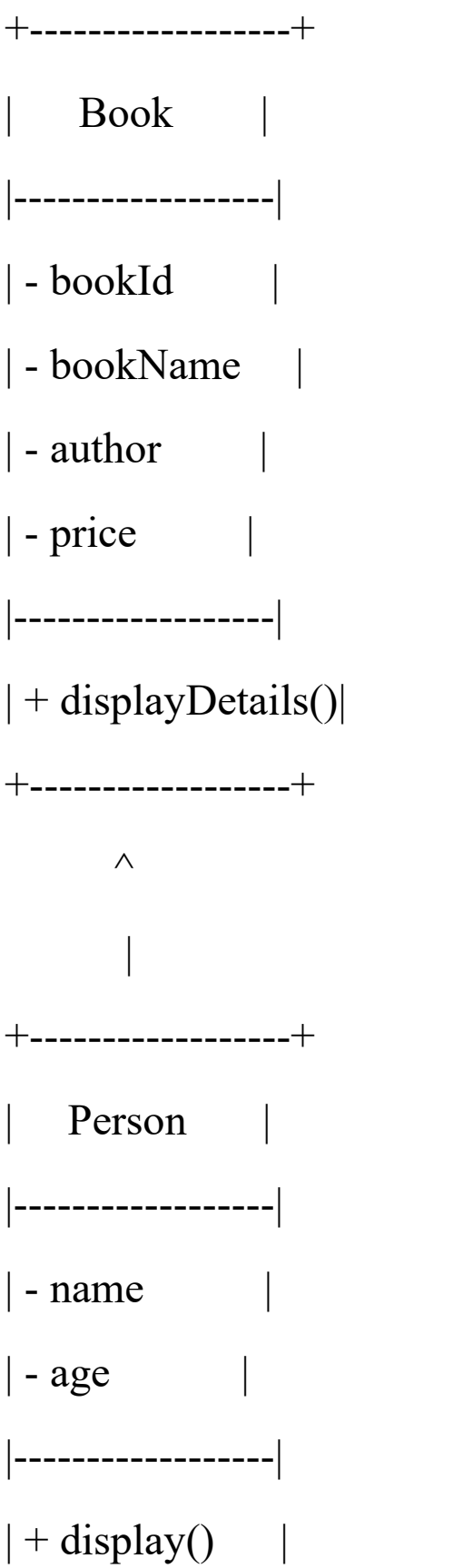
Steps for Execution

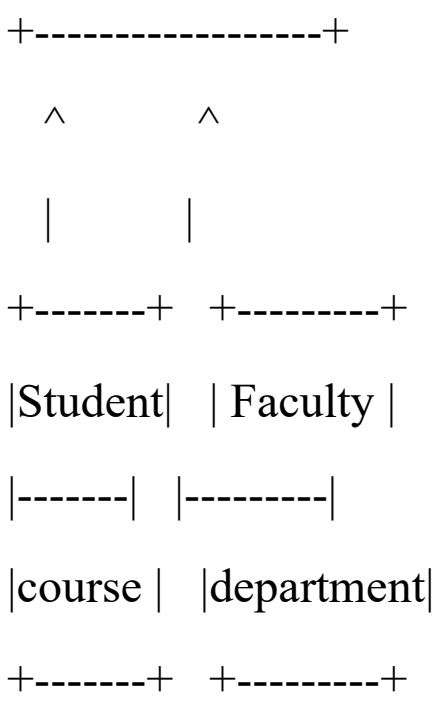
1. Open your IDE (Eclipse/VS Code/IntelliJ).
2. Create a new Java project named LibraryManagementSystem.
3. Add the classes: Book, Person, Student, Faculty, AreaCalculator, Vehicle, Car, Bike, Shape, Circle, Rectangle, Printable, Report.
4. Write the main() method inside the LibraryManagementSystem class.
5. Save the file as LibraryManagementSystem.java.
6. Compile the program: javac LibraryManagementSystem.java
7. Run the program: java LibraryManagementSystem
8. Observe the output on the console.

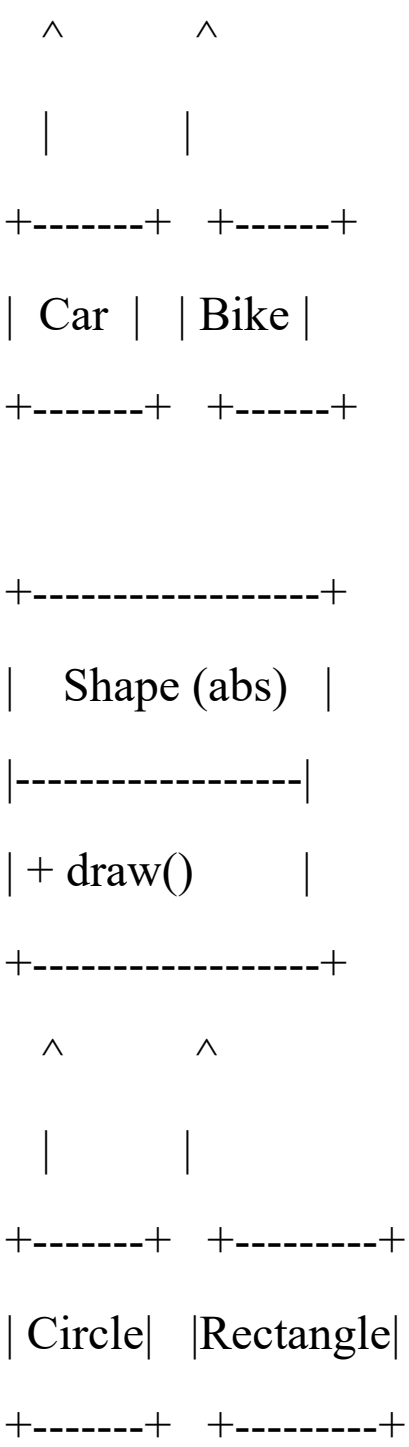
Algorithm / Logic Description

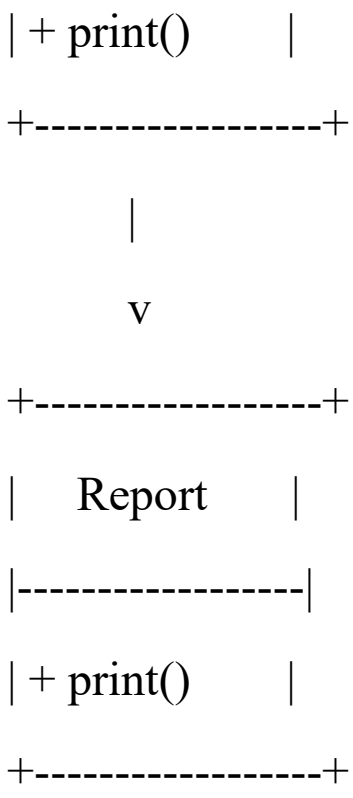
1. Define a Book class with private variables, constructors, getters/setters, and a display method (**Encapsulation**).
2. Define a Person superclass and extend it with Student and Faculty subclasses (**Inheritance**).
3. Define an AreaCalculator class with overloaded area() methods, and a Vehicle class with overridden display() methods in Car and Bike (**Polymorphism**).
4. Define an abstract Shape class with abstract method draw(), and implement it in Circle and Rectangle (**Abstraction**).
5. Define an interface Printable with method print(), and implement it in Report (**Interfaces**).
6. In main(), create objects of each class and call their methods to demonstrate the concepts.
7. Display the results on the console.

UML Class Diagram




+------------------+
^ ^
| |
+-------+ +---------+
|Student| | Faculty |
|-------| |---------|
|course | |department|
+-------+ +---------+
+------------------+
AreaCalculator
+ area()
+------------------+
+------------------+
Vehicle
+ display()
+------------------+


Car
Bike
Shape (abs)
+ draw()
Circle
Rectangle
Printable (intf)



Source Code

```
package librarysystem;
class Book {
   private int bookId;
   private String bookName;
   private String author;
   private double price;

   // const
   Book() {
      bookId = 0;
```

```
        bookName = "Unknown";
        author = "Unknown";
        price = 0.0;
    }
   //const with param
    Book(int id, String name, String auth, double pr) {
        bookId = id;
        bookName = name;
        author = auth;
        price = pr;
    }

    // getters & setters for Encapsulation
    public int getBookId() { return bookId; }
    public void setBookId(int id) { bookId = id; }

    public String getBookName() { return bookName; }
    public void setBookName(String name) { bookName =
name; }

    public String getAuthor() { return author; }
```

```
    public void setAuthor(String auth) { author = auth; }

    public double getPrice() { return price; }
    public void setPrice(double pr) { price = pr; }

    public void displayDetails() {
        System.out.println(bookId + " | " + bookName + " | " +
author + " | " + price);
    }
}
 // inheritance super:person sub:student and faculty
class Person {
    String name;
    int age;
    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

class Student extends Person {
    String course;
```

```
    void display() {
        super.display();
        System.out.println("Course: " + course);
    }
}

class Faculty extends Person {
    String department;
    void display() {
        super.display();
        System.out.println("Department: " + department);
    }
}

//polymorph
class AreaCalculator {
    double area(double radius) {
        return Math.PI * radius * radius;
    }
  double area(double length, double breadth) {
        return length * breadth;
```

```
    }
    double area(double base, double height, boolean
triangle) {
        return 0.5 * base * height;
    }
}

class Vehicle {
    void display() {
        System.out.println("This is a vehicle.");
    }
}

class Car extends Vehicle {
    @Override
    void display() {
        System.out.println("This is a Car.");
    }
}

class Bike extends Vehicle {
```

```
    @Override
    void display() {
        System.out.println("This is a Bike.");
    }
}

// abstraction
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a Circle");
    }
}

class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
```

```
}

interface Printable {
   void print();
}

class Report implements Printable {
   public void print() {
      System.out.println("Printing Report...");
   }
}



public class LibrarySystem {
        public static void main(String[] args) {
      System.out.println(" Book Details :");
      Book b1 = new Book();
      Book b2 = new Book(235, "Java programming
practices", "Santosh Kumar P", 499.99);
      b1.displayDetails();
```

```
        b2.displayDetails();

        System.out.println("\n Inheritance:");
        Student s = new Student();
        s.name = "Haarika";
        s.age = 20;
        s.course = "Computer Science";
        s.display();

        Faculty f = new Faculty();
        f.name = "Dr. Rao";
        f.age = 45;
        f.department = "Engineering";
        f.display();

        System.out.println("\nPolymorphism:");
        AreaCalculator ac = new AreaCalculator();
        System.out.println("Circle Area: " + ac.area(5));
        System.out.println("Rectangle Area: " + ac.area(4, 6));
        System.out.println("Triangle Area: " + ac.area(3, 7,
true));
```

```
        Vehicle v = new Vehicle();
        Vehicle car = new Car();
        Vehicle bike = new Bike();
        v.display();
        car.display();
        bike.display();

        System.out.println("\n Abstraction & Interfaces");
        Shape circle = new Circle();
        Shape rectangle = new Rectangle();
        circle.draw();
        rectangle.draw();
        Report report = new Report();
        report.print();
    }

}

//Observation Questions (Simple Answers)
```

//1. What is Object-Oriented Programming (OOP)?

//OOP is a way of writing programs using objects.

//Objects have data like variables and actions like methods

//It makes programs easier to manage and reuse.

//2. Difference between Class and Object.

//Class is like a design or blueprint.

//Object is the real thing made from that design.

//Example: Book is a class, b1 = new Book() is an object.

//3. Purpose of Constructors.

//Constructors are used to give starting values to objects like to initialize variables

//They run automatically when we create an object in default even if we dont write in code

//Default constructor gives default values, parameterized constructor takes values from user like in code we saw their performance differences

//4. What is Encapsulation? How in Java?

//Encapsulation means keeping variables safe inside a classan d give access where and who can use or cant use variables of a particular class

//In Java we make variables private and use getter/setter methods to access them as we can see in above code

//5. Difference between Method Overloading and Method Overriding.

//Overloading: Same method name but different parameters

//Overriding: Same method name and parameters but different code (done in child class).

//6. Four pillars of OOP with examples.

//Encapsulation: Book class with private variables and getters/setters.

//Inheritance: Student and Faculty classes extend Person.

//Polymorphism: area() method used for circle, rectangle, triangle (overloading) and display() in Vehicle/Car/Bike (overriding).

//Abstraction: Shape abstract class with draw() method, implemented in Circle and Rectangle.

Output Screenshot:

Book Details:

0 | Unknown | Unknown | 0.0

101 | Java Programming Practices | Santosh Kumar P | 499.99

Inheritance:

Name: Haarika, Age: 20

Course: Computer Science

Name: Dr. Santosh, Age: 45

Department: Engineering

Polymorphism:

Circle Area: 78.53981633974483

Rectangle Area: 24.0

Triangle Area: 10.5

This is a vehicle.

This is a Car.

This is a Bike

Abstraction & Interfaces:

Drawing a Circle

Drawing a Rectangle

Printing Report...